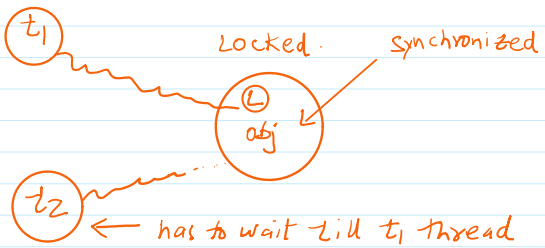
Deadlock

08 April 2026 18:34

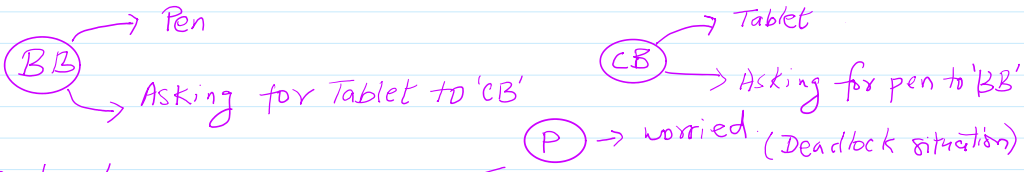
If two threads are waiting for each other forever, such type of infinite wating is called deadlock.

"synchronized" keyword is the only reason for deadlock situation, hence by using "synchronized" keyword we have to take special care.



There is no resoultion technique for deadlock. But prevention techniques are available.

Prevention technique:- never call another synchronized method from a synchronized method, if the case is happening revise the logic of program design.



'BB' & 'CB' both wait forever. This is the deadlock.

Conditions for Deadlock

1. Mutual Exclusion:- Resource can be used by one thread at a time.
2. Hold and wait:- Thread holds one resource and waits for another.
3. No Preemption:- Resource cannot be forcibly taken.
4. Circular wait:- Threads wait in a circular chain.

```
package ThreadSynchronization;
public class DeadLockOne {
    static final Object resource1 = new Object();
    static final Object resource2 = new Object();
    public static void main(String[] args) {
        Thread t1 = new Thread(()->{
            synchronized(resource1){
                System.out.println("Thread 1: locked resource-1" );
                try {
                    Thread.sleep(10000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                synchronized(resource2){
                    System.out.println("Thread-1: locked resource-2");
                }
            }
```

```
        }
        );
        Thread t2 = new Thread(()->{
            synchronized(resource2){
                System.out.println("Thread-2: locked resource-2");
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                synchronized(resource1){
                    System.out.println("Thread 1: locked resource-1");
                }
            }
        });
        t1.start();
        t2.start();
    }
}
```

Output:

```
Thread 1: locked resource-1
Thread-2: locked resource-2
```

Infinitely code is executing.

How deadlock occurs in this Program:

1. Thread-1 locks resource-1
2. Thread-2 locks resource-2
3. Thread-1 waits for resource-2
4. Thread-2 waits for resource-1
5. Both waits forever → Deadlock

To detect deadlock Java provides ThreadMXBean

public interface ThreadMXBean extends PlatformManagedObject

The management interface for the thread system of the Java virtual machine.
ThreadMXBean supports monitoring and management of platform threads in the Java virtual machine. Platform threads are typically mapped to kernel threads scheduled by the operating system. ThreadMXBean does not support monitoring or management of virtual threads.
A Java virtual machine has a single instance of the implementation class of this interface. This instance implementing this interface is an MXBean that can be obtained by calling the ManagementFactory.getThreadMXBean() method or from the platform MBeanServer method.
The ObjectName for uniquely identifying the MXBean for the thread system within an MBeanServer is:
java.lang:type=Threading
It can be obtained by calling the PlatformManagedObject.getObjectName() method.

Synchronization Information and Deadlock Detection
Some Java virtual machines may support monitoring of object monitor usage and ownable synchronizer usage. The getThreadInfo(long[], boolean, boolean) and dumpAllThreads(boolean, boolean) methods can be used to obtain the thread stack trace and synchronization information including which *Lock* a thread is blocked to acquire or waiting on and which locks the thread currently owns.
The ThreadMXBean interface provides the findMonitorDeadlockedThreads() and findDeadlockedThreads() methods to find deadlocks in the running application.
Since:

1.5

From <https://docs.oracle.com/en/java/javase/26/docs/api/java.management/java/lang/management/ThreadMXBean.html>

```
package ThreadSynchronization;
public class DeadLockDetect implements Runnable{
    final Object resource1;
    final Object resource2;
    public DeadLockDetect(Object lock1, Object lock2){
        this.resource1 = lock1;
        this.resource2 = lock2;
    }
    @Override
    public void run() {
        synchronized(resource1){
            System.out.println(Thread.currentThread().getName()+": acquired lock");
            try {
                Thread.sleep((1000));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            synchronized(resource2){
                System.out.println(Thread.currentThread().getName()+": acquired lock");
            }
        }
        synchronized(resource2){
            System.out.println(Thread.currentThread().getName() + ": acquired lock");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            synchronized(resource1){
                System.out.println(Thread.currentThread().getName()+": acquired lock");
            }
        }
    }
}
```

```
package ThreadSynchronization;
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;
public class DeadLockDetectMX{
    public static void deadLockDetect(){
        ThreadMXBean bean = ManagementFactory.getThreadMXBean();
        long[] ids = bean.findDeadlockedThreads();
        if(ids != null){
            System.out.println("DeadLock detected!");

            ThreadInfo[] infos = bean.getThreadInfo(ids, true, true);

            for (ThreadInfo threadInfo : infos) {
                System.out.println("Thread name: " + threadInfo.getThreadName());
                System.out.println("Thread State: " + threadInfo.getThreadState());
                System.out.println("Locked On: " + threadInfo.getLockName());
                System.out.println("Owned by: "+threadInfo.getLockOwnerName());
                System.out.println("Stack trace: ");
                for (StackTraceElement ste: threadInfo.getStackTrace()) {
                    System.out.println("\t"+ ste);
                }
                System.out.println("------------------------------------------------");
            }
        }
        else{
            System.out.println("No deadlock detected!");
```

scans JVM finds deadlocked thread IDs

gets stack trace & locked resource information, & the owners information is returned.

```
        }
    }
    public static void main(String[] args) {
        Object r1 = new Object();
        Object r2 = new Object();
        Thread t1 = new Thread(new DeadLockDetect(r1, r2), "Ajay");
        Thread t2 = new Thread(new DeadLockDetect(r1, r2), "Vijay");
        t1.start();
        t2.start();
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        DeadLockDetectMX.deadLockDetect();
    }
}
```

These two threads locks resource in opposite order. & Both wait for each other's lock.

this code executes because Main thread is not the part of deadlock.

Output:

```
Ajay: acquired lock
Ajay: acquired lock
Ajay: acquired lock
Vijay: acquired lock
DeadLock detected!
Thread name: Ajay
Thread State: BLOCKED
Locked On: java.lang.Object@5a07e868
Owned by: Vijay
Stack trace:
    app//ThreadSynchronization.DeadLockDetect.run(DeadLockDetect.java:34)
    java.base@24.0.2/java.lang.Thread.runWith(Thread.java:1460)
    java.base@24.0.2/java.lang.Thread.run(Thread.java:1447)
-------------------------------------------------
Thread name: Vijay
Thread State: BLOCKED
Locked On: java.lang.Object@27c170f0
Owned by: Ajay
Stack trace:
    app//ThreadSynchronization.DeadLockDetect.run(DeadLockDetect.java:22)
    java.base@24.0.2/java.lang.Thread.runWith(Thread.java:1460)
    java.base@24.0.2/java.lang.Thread.run(Thread.java:1447)
-------------------------------------------------
```

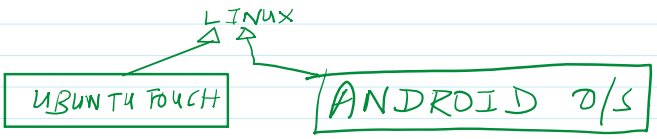
Important Notes:

Works for:

- synchronized locks
- Reentrant lock
- Ownable synchronizers

Return

- null -> No deadlock
- long[] -> Thread ID's involved

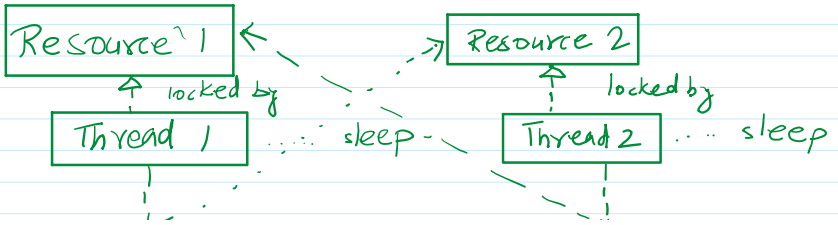


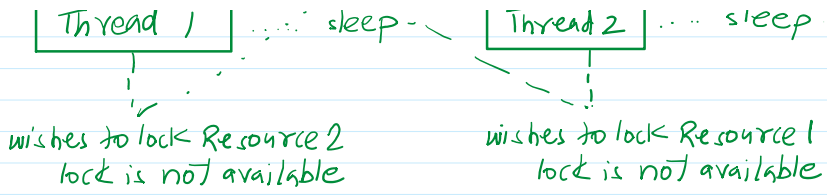
Summary:

ThreadMXBean provides a programmatic way to:

- Detect deadlocks
- Inspect involved threads
- Get stack traces
- Analyze root cause

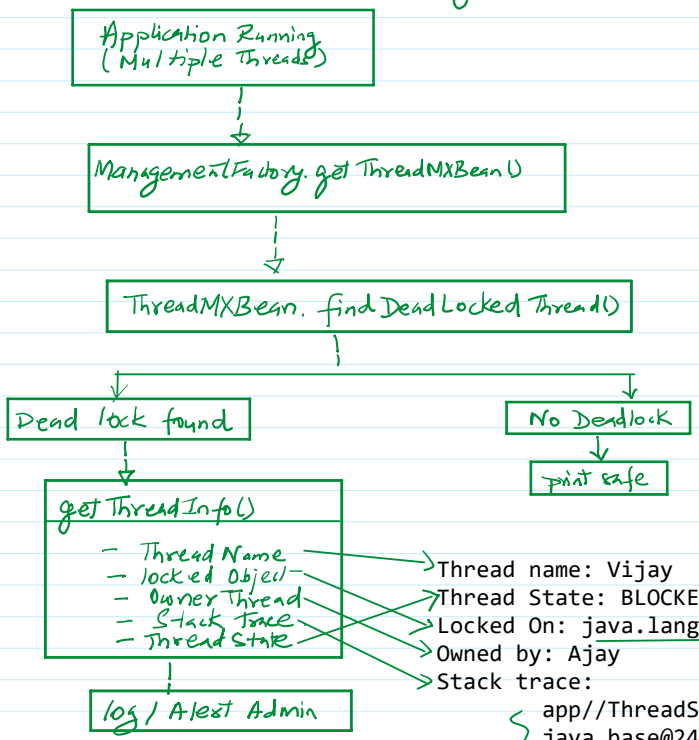
This is a production grade debugging techniques used in real JVM monitoring system.





No thread can proceed, circular dependency is formed and program freezes.

Deadlock detection using ThreadMXBean – Flow diagram.




```
Thread name: Vijay
Thread State: BLOCKED
Locked On: java.lang.Object@27c170f0
Owned by: Ajay
Stack trace:
        app//ThreadSynchronization.DeadLockDetect.run(DeadLockDetect.java:22)
        java.base@24.0.2/java.lang.Thread.runWith(Thread.java:1460)
        java.base@24.0.2/java.lang.Thread.run(Thread.java:1447)
-----------------------------------------------
```